

# **CH2 - Clip 17.**

## **Hyper-Parameter Tuning**



01

**Hyper  
Parameter  
Tuning**

02

**방법론**

03

**일반적인  
Tuning 작업  
흐름**

# 1. Hyper-Parameter Tuning

# Hyper-Parameter Tuning

## 정의

- 머신러닝 모델의 성능을 최적화하기 위해 사용하는 Hyper-parameter들의 값을 조정하는 과정
- Hyper-parameter란?  
Model Training 중에 자동으로 조정(학습)되지 않는 모델의 Parameter를 의미하며, 학습 전에 사전에 지정되어야함

## 예시

- Neural Networks
  - Learning rate : 가중치 업데이트의 크기를 결정
  - Batch Size : 한 번의 배치에서 학습하는 데이터 수
  - Number of Layers : Neural의 layer
  - Number of Units per layers : 각 층의 neuron / unit number
- Decision Trees
  - Maximum depth : Tree의 최대 깊이
  - Minimum samples split : node를 분할하기 위한 최소 샘플 수
- Support Vector Machines
  - C (regularization parameter) : 오류 항에 대한 penalty
  - Kernel : 사용되는 커널 타입

# Hyper-Parameter Tuning

## 중요성

- 올바른 설정은 모델의 성능에 큰 영향
- 너무 낮거나 높은 learning rate, 적절하지 않은 tree depth 등은 모델 학습 능력을 제한하거나 overfitting을 초래할 수 있음
- 적절한 hyper-parameter 값을 찾는 것은 매우 중요한 단계

## 방법론

- Grid-Search
- Random-Search
- Bayesian Optimization
- Gradient-based Optimization
- Evolutionary Algorithms

## 2. 방법론

# 방법론 : Grid Search

## 정의

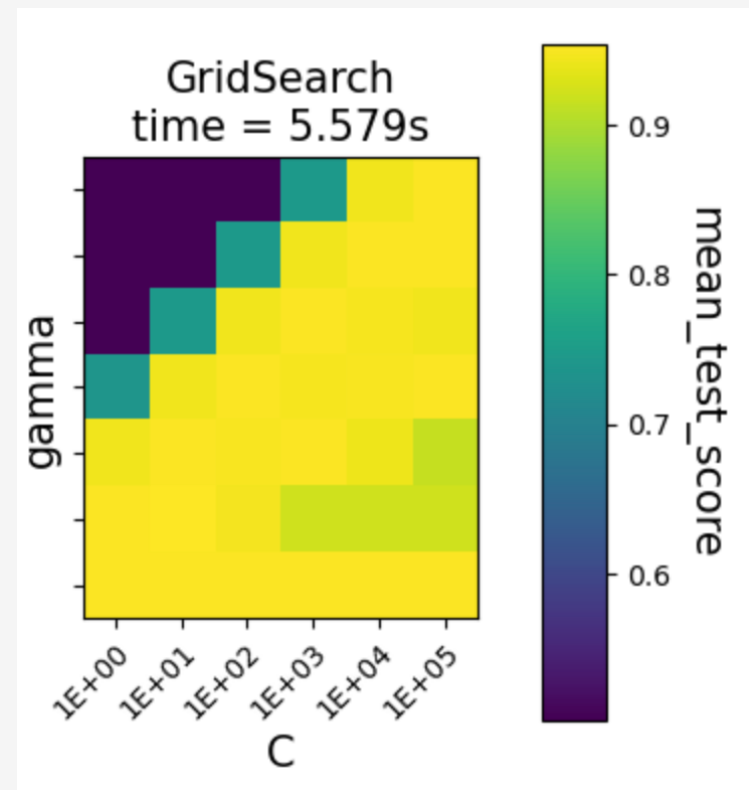
- Hyper-parameter들의 가능한 모든 조합을 탐색하는 방식
- 사용자는 각 hyper-parameter에 대한 가능한 값을 명시적으로 제공하고, Grid-Search는 이러한 조합의 모든 가능한 세트를 시도하여 최적의 성능 얻음

## 작동 원리

1. 각 hyper-parameter에 대한 탐색 범위 또는 값의 집합을 지정
2. 모든 가능한 hyper-parameter 조합을 생성
3. 각 조합에 대해 모델을 학습시키고, 성능 지표를 사용하여 평가
4. 가장 성능이 좋은 조합을 선택

## 특징

- 장점 : 모든 가능한 조합을 고려하기 때문에 최적의 값을 놓치지 않음. 재현 가능함
- 단점 : 탐색 범위가 클 경우 시간과 연산 비용이 높음



출처 : scikit-learn.org

# 방법론 : Random Search

## 정의

- Hyper-parameter 값들을 무작위로 선택하여 탐색하는 방식
- 각 Hyper-parameter에 대한 분포를 지정하고, Random Search는 이 분포에서 값을 Sampling하여 모델을 평가

## 작동 원리

1. 각 hyper-parameter에 분포를 지정
2. 지정된 반복 횟수 또는 시간 동안 분포에서 무작위로 값을 Sampling
3. Sampling된 값으로 모델을 학습시키고, 성능 지표를 사용하여 평가
4. 가장 성능이 좋은 조합을 선택

## 특징

- 장점 : Grid Search보다 비용이 낮음. 더 다양한 조합의 값을 빠르게 탐색
- 단점 : 최적의 값이 탐색 공간에 있더라도 Random으로 놓칠 수 있음. 결과 재현성이 보장 안됨

# 방법론 : Bayesian Optimization

## 정의

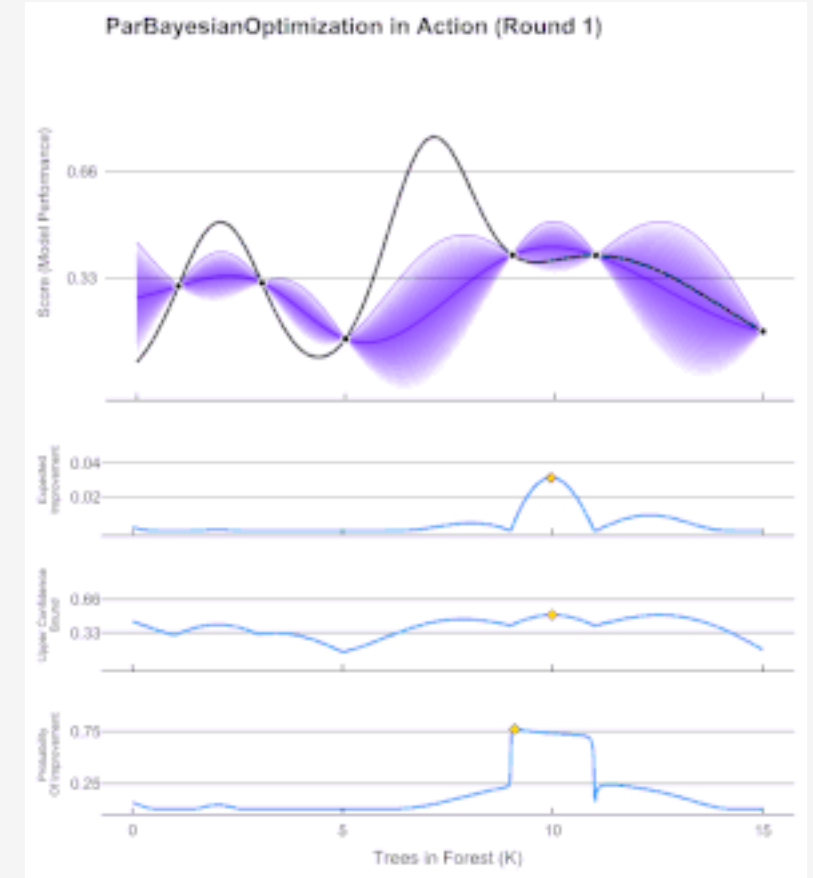
- 확률 모델을 기반으로 hyper-parameter의 최적 조합을 탐색하는 방식
- 성능 불확실성을 모델링하고, 이 정보를 사용하여 다음에 시도할 hyper-parameter 조합을 결정

## 작동 원리

1. Gaussian Process와 같은 확률 모델로 함수의 불확실성을 모델링
2. Acquisition function을 사용하여 탐색할 위치를 결정
3. 선택된 위치에서 함수를 평가하고 확률 모델을 업데이트
4. 이 과정을 반복

## 특징

- 장점 : 연산 비용이 높은 함수에 효과적, 탐색 과정에서의 불확실성을 고려
- 단점 : 모델링 과정이 복잡. 높은 차원의 탐색 공간에서 덜 효과적



출처 : wikipedia

# 방법론 : Gradient-based Optimization

## 정의

- Hyper-parameter의 gradient 정보를 사용하여 최적의 값을 탐색하는 방식
- 최근에는 자동 미분과 같은 기술이 발전해서 hyper-parameter 최적화에 gradient 기반 접근법이 사용되기 시작

## 작동 원리

1. Hyper-parameter 초기 값에서 시작
2. 해당 값에서 gradient 값을 계산
3. Gradient 방향으로 hyper-parameter를 업데이트
4. 최적의 값을 찾을 때까지 이 과정을 반복

## 특징

- 장점 : 연속형 hyper-parameter에 효과적
- 단점 : gradient를 계산할 수 없는 경우나 불연속적은 hyper-parameter에 적용하기 어려움. Local optimization에 갇힐 수 있음

# 방법론 : Evolutionary Algorithms

## 정의

- 자연 선택의 원리를 모방하여 hyper-parameter의 최적 조합을 탐색하는 방식
- 개체(hyper-parameter 조합)의 세대를 거쳐 가장 적합한 개체들이 다음 세대로 선택되고 변화

## 작동 원리

1. 초기 개체 집합(모델의 hyper-parameter 조합)을 생성
2. 각 개체의 적합도를 평가
3. 선택, 교차, 돌연변이 연산을 사용하여 새로운 개체를 생성
4. 이 과정을 반복하여 최적의 개체 찾을

## 특징

- 장점 : 여러 해를 동시에 탐색 가능
- 단점 : 연산 비용이 높고, 알고리즘의 hyper-parameter를 또 설정해야함

# Summary

## Summary

- 다양한 model tuning 방식을 통해 모델을 최적화 필요
- 이런 다양한 반복과 복잡한 과정을 MLOps에 다루어 줘야함